

M1.1: eBPF 内核程序类型与加载流程

Theory: 贯穿内核协议栈各层面的可编程动态挂载。eBPF 允许将自定义字节码挂载到内核特定网络钩子。

Details: Cilium 核心使用 ‘XDP’ 进行驱动级高速拦截, ‘tc-bpf’ 挂载在网络流量控制层 (Ingress/Egress) 以处理容器网络包, 以及 ‘sockops’ 挂载在套接字协议栈以短路同机连接。Cilium Agent 通过系统调用 ‘bpf()’, 在内核各锚点动态载入字节码, 并锁固于虚拟文件系统 ‘sys/fs/bpf’。

Trade-off: 加载 eBPF 涉及内核锁, 高频加载会导致宿主机短暂 CPU 卡顿。因此, 程序仅在网络接口变动或升级时触发加载, 其余逻辑采用 BPF Maps 动态路由。

M1.4: CO-RE 与 BTF 重定位机制

Theory: 解决跨 Linux 内核版本兼容性问题的 “一次编译, 处处运行 (Compile Once, Run Everywhere)” 机制。

Details: 传统 eBPF 编译物深度依赖内核结构体成员的物理内存偏移量。Cilium 引入 CO-RE, 配合 BTF (BPF Type Format) 类型元数据。在程序加载时, Libbpf 对比当前系统内核的 BTF, 动态修正并改写指令中的结构体成员偏移, 确保跨内核安全执行。

Trade-off: 要求内核必须开启 ‘CONFIG_DEBUG_INFO_BTF’ 选项。在非常老旧的系统上, CO-RE 无法启用, Cilium 被迫回退到在宿主机上现场编译二进制的笨重方案。

M2.1: XDP 网卡驱动层极速拦截

Theory: 绕过内核几乎全部网络协议栈的极速包处理技术。

Details: XDP 挂载在物理网卡驱动的 DMA 环形缓冲区接收阶段。当网卡中断发生时, XDP 字节码在尚未为数据包创建复杂的 ‘sk_buff’ 内存结构前, 直接读取物理数据帧。通过返回 ‘XDP_DROP’ 丢弃垃圾包, 或者 ‘XDP_REDIRECT’ 从其他网卡物理发回。

Trade-off: XDP 运行得太早, 使得其无法直接使用 TCP 协议栈等内核高级功能, 且需要网卡驱动原生支持 (Native XDP)。如果驱动不支持, 只能回退到有内存开销的 Generic 模拟模式。

M2.2: tc (Traffic Control) 接口钩子

Theory: 运行在 sk_buff 阶段的协议栈级双向流量拦截。

Details: tc-bpf 挂载在 Linux 的 Traffic Control 调度层, 拥有 Ingress 和 Egress 两个方向。相比 XDP, 它能够读取完整的 ‘sk_buff’ 结构, 获取网络接口索引、包元数据 (skb->mark), 支持更丰富的三层路由和包头重写指令。

Trade-off: 相比 XDP 略微滞后, 会产生 ‘sk_buff’ 的内存分配损耗。Cilium 使用 tc 作为核心转发引擎, 将 XDP 仅用于防 DDOS 过滤和高速负载均衡。

M1.2: LLVM/Clang eBPF 机器码生成

Theory: 标准 C 语言子集编译为受限内核虚拟指令集的流水线。

Details: Clang/LLVM 编译器接收限制性 C 代码, 生成 64 位 eBPF 字节码 (eBPF ISA)。由于内核运行的安全约束, 代码不能包含任意循环 (必须在编译期完全展开)、不能有危险的局部变量指针逃逸, 且编译出的指令深度受到内核严格限制。

Trade-off: 不支持标准 C 库函数, 必须使用特定的 ‘bpf_helper_x’ 辅助函数。复杂的算子需要在应用层用 Go/Rust 完成, eBPF 仅保留网络包路由和过滤核心逻辑。

M1.5: BPF Map 类型与内核哈希表

Theory: 用户态进程与内核态 eBPF 代码、以及不同 eBPF 钩子之间共享和存储状态无锁共享内存。

Details: BPF Maps 是内核维护的键值存储结构。Cilium 深度使用 ‘BPF_MAP_TYPE_HASH’ 存储连接追踪表, ‘BPF_MAP_TYPE_LRU_HASH’ 实现会话保持, ‘BPF_MAP_TYPE_LPM_TRIE’ 进行 IP 最长前缀匹配。读写操作通过 ‘bpf_map_lookup_elem’ 等无锁原子函数执行。

Trade-off: BPF Map 大小是在加载时静态分配的。若突发超大规模网络并发导致连接追踪哈希表满, 会发生丢包。Cilium 会预估集群规模, 预先分配百万级槽位并在后台启动垃圾清理线程。

M1.3: libbpf 与 bpftool 内核验证安全防线

Theory: 防止用户态恶意或有缺陷的代码把 Linux 内核跑死或写崩溃的物理沙箱边界安全验证器。

Details: 在 eBPF 字节码提交给内核时, 内核的 ‘Verifier’ (验证器) 会对代码进行静态控制流分析。它确保所有跳转不形成死循环、寄存器读写边界合法、指针没有野指针解引用。‘libbpf’ 负责打包和重定位, ‘bpftool’ 用于动态观测内核中活跃的 BPF 结构。

Trade-off: 验证器过于严苛, 会导致稍微复杂的 C 逻辑直接被拒绝加载。编写代码时, 必须使用类似 ‘if (ptr + 1 > limit) return;’ 的防御性代码显式向验证器证明指针安全。

M1.6: Cilium Agent 动态编译流

Theory: 运行期根据 K8s 网络拓扑和安全策略动态生成 BPF 目标代码的 “代码生成器”。

Details: Cilium Agent 是运行在 K8s 节点上的守护进程。当发现新 Pod 创建或 NetworkPolicy 变更时, Agent 会动态生成包含特定常量 (如 Pod IP、网络接口索引) 的 C 语言头文件, 并在后台并发调用 ‘clang’ 进行 JIT 般编译生成 ‘o’ ELF, 通过系统调用热替换内核程序。

Trade-off: 在超大规模集群突发扩容时, 并发编译 Clang 会吃光 CPU。Cilium 引入了静态机器码配制化 (Static Caching) 与编译去抖, 大幅精简热加载编译耗时。

M2.3: eBPF 数据包头无锁解析

Theory: 基于指针算术的高效包头协议边界提取。

Details: eBPF 程序接收指向数据包头和尾物理内存的两个裸指针 (‘data’ 和 ‘data_end’)。解析 IP 或 TCP 头时, 直接用指针转换 (Pointer Casting) 进行偏移偏移。例如, 判断是否为 IPv4: ‘(struct iphdr *) (data + eth_header_len)’。每次访问前, 必须有代码向验证器证明边界不会越界: ‘if (ip_ptr + 1 > data_end) return tc_drop;’。

Trade-off: 指针安全边界验证极其繁琐, 稍有疏漏会导致编译失败。代码中充斥着大量的边界防御条件判断。

M2.4: Conntrack 连线跟踪 Map 机制

Theory: eBPF 空间内维护的状态机连接表, 用于记录每个 TCP/UDP 流量双向连接关系。

Details: Cilium 在 BPF 中实现了一个完全脱离内核 Conntrack 的状态表。以 (‘源 IP, 源端口, 目的 IP, 目的端口, 协议’) 作为 Key 写入哈希 Map。包经过 tc Ingress 时, 如果发现是 TCP SYN 且安全策略允许, 则写入连接项; 随后的回应包可通过 Map 查找快速放行。

Trade-off: 需要自行处理连接超时 (如半连接超时、长时间无心跳清理)。Cilium Agent 通过在后台常驻定时扫描线程, 定期清理 Map 中的僵尸连接, 避免内存耗尽。

M2.5: eBPF 内核 NAT 与三层路由重写

Theory: 内核空间内的高速包头字段改写与单系统调用重写转发。

Details: 实现负载均衡或 SNAT 时，eBPF 程序直接修改 ‘sk_buff’ 中的 IP 和端口字段。修改字段后，必须更新 L3/L4 校验和 (Checksum)，使用 ‘bpf_l3_csum_replace’ 重新解算。随后，调用 ‘bpf_redirect’ 直接将包送入对应虚拟/物理网卡驱动发送，跳过物理路由查找表。

Trade-off: 硬件校验和卸载 (Checksum Offloading) 失效可能会造成物理丢包。Cilium 会智能识别网卡特性，在不支持卸载的设备上通过 BPF 代码强制在 CPU 上全量重算。

M3.2: SOCKMAP 描述符绑定哈希表

Theory: BPF 中专门用于存储活跃 Socket 物理引用以便进行快速路由的特殊 Map 结构。

Details: Cilium 声明一个 ‘BPF_MAP_TYPE_SOCKMAP’。在 sockops 程序捕获到 TCP 连接建立事件时，如果判断源 IP 和目的 IP 都在本台宿主机上，则以 ‘(IP, 端口, 目的 IP, 目的端口)’ 为 Key，将当前 Socket 的引用直接写入 SOCKMAP。

Trade-off: SOCKMAP 必须在内核关闭连接 (TCP FIN/RST) 时自动清理，否则会产生野指针。Cilium 挂载特定的 sockops 回调事件来监听连接生命周期并在 map 中擦除。

M3.5: Socket 锁竞争与系统调用消除

Theory: 通过减少多核 CPU 下网络操作锁争抢和减少内核切换来提升并发服务极限性能。

Details: 传统的 Socket 操作会在数据进入网络层时频繁争抢 Socket 锁，且数据在用户态/内核态不断拷贝。通过 eBPF 短路，仅在 ‘sendmsg’ 阶段在内核中单次将内存数据重定向到接收队列，消除了后续网络层和传输层的锁争用和软件中断过程。

Trade-off: 此种极致短路目前主要支持 TCP，对于 UDP 等无连接协议的短路，效果和逻辑实现均较为有限。

M4.2: 负载均衡虚 IP (VIP) 映射

Theory: 在内核入口将 K8s Service VIP 动态重写为后端 Real Pod IP。

Details: Cilium 在 tc ingress 或 XDP 中拦截进入的数据包。如果发现目的 IP 匹配某个虚拟 IP，则根据 BPF Map 中的 Service 服务表执行 Maglev 路由，动态改写数据包的 Dst IP 为具体 Real Pod IP，同时记录 Session。

Trade-off: VIP 映射改变了数据包的目的地址，因此返回包到达网关时必须进行逆向 NAT (将 Source IP 重写回 VIP)，否则客户端会因为源地址对不上而直接拒绝连接。

M2.6: VxLAN/Geneve 隧道封装与解包

Theory: 云原生网络覆盖 (Overlay) 模式下，跨节点传输时的无缝包嵌套。

Details: 跨宿主机通信时，tc-bpf 提取原始数据包，在头部硬编码拼接上 VxLAN/Geneve 头和外部宿主机外层 IP。当外部包到达目标节点时，tc Ingress 程序拦截，识别隧道标识符 (VNI)，剥离外层 IP 和隧道头，还原原始数据包并转发给目标 Pod。

Trade-off: 隧道封装会增加约 50 字节的额外开销，这可能导致网络传输突破物理 MTU 大小从而产生 IP 分片 (Fragmentation) 导致性能暴跌。必须调整容器网卡的 MTU 大小为 1450。

M3.3: bpf_msg_redirect_hash 缓冲区短路

Theory: 本地网络 IO 的“虫洞”技术。将发送 Socket 的输出缓冲区直接倒入接收 Socket 的输入缓冲区。

Details: 挂载在 ‘BPF_PROG_TYPE_SK_MSG’。当 Pod A 通过 Socket 写入数据并调用 ‘sendmsg’ 时，该 BPF 程序在应用层数据刚进入套接字发送缓冲 (sk_send_head) 阶段进行拦截，查询 SOCKMAP 找到同机 Pod B 的接收 Socket，调用 ‘bpf_msg_redirect_hash’ 辅助函数直接将数据块注入 Pod B 的 ‘sk_receive_queue’，完全避开了 TCP 协议栈封包、物理回环设备等层层传递。

Trade-off: 此机制使数据包在协议栈中“隐形”，导致主机上经典的 ‘tcpdump’ 抓包工具在回环网卡上完全捕获不到数据，需要依赖 Hubble 探针进行元数据捕获。

M3.6: Unix Domain Socket 性能对比

Theory: 微服务架构下采用 TCP 加速还是重构为 Unix Domain Socket 的决策模型。

Details: 虽然 UDS 比传统 TCP 快，但在微服务下，强行将代码中的 ‘http://ip:port’ 改造为 ‘unix:///path’ 会涉及巨大的重构成本。Cilium 的 TCP 短路使得应用层不需要做任何代码修改，依然访问正常的 TCP 套接字，而在内核层却获得了媲美 UDS 的物理传输性能。

Trade-off: 在极小数据量的高频交互下，UDS 依然因无三次握手等控制开销而略微占优。对于超大吞吐网络，Cilium 短路后性能差异基本被抹平。

M4.3: BPF LRU Map 会话保持

Theory: 在高并发连接吞吐下，对持续会话状态进行高性能查找并能自动清理过载数据的无锁设计。

Details: 为了确保同一客户端的后续网络包一直落到同一个后端 Pod，Cilium 将 ‘(ClientIP, ServiceVIP) -> PodIP’ 的映射记录在 ‘BPF_MAP_TYPE_LRU_HASH’ 中。LRU (最近最少使用) 算法由内核 BPF 直接支持，在 Map 满时自动驱逐冷数据，防止内存泄露。

Trade-off: LRU 的数据驱逐是概率性的，在高频并发冲突下可能会出现极低概率的冷连接会话保持失效，必须在应用层设计好重试机制。

M3.1: sockops 套接字层事件拦截

Theory: 在 Socket 建立连接阶段将套接字上下文捕获进可编程内核的管理技术。

Details: 挂载在 ‘BPF_PROG_TYPE_SOCK_OPS’。当进程调用 ‘connect()’、‘accept()’ 完成 TCP 握手时，此 BPF 程序被自动触发。它能够读取当前 Socket 的源 IP、源端口、目的 IP、目的端口等元组，并获取当前 Socket 的物理上下文指针。

Trade-off: sockops 主要适用于本地同机 TCP 加速。对于跨机连接，写入额外的映射会产生极微小的物理内存浪费。

M3.4: 同机 Pod 间本地通信加速折中

Theory: 用可观测性与传统路由审计死角为代价换取吞吐爆发的折中设计。

Details: 传统同机通信需要走完整的 TCP 状态机、系统调用、软中断和网卡队列。Cilium sockops 直接让字节跨 Socket 拷贝，吞吐性能提升数倍，延迟暴跌。

Trade-off: 绕过了内核 Netfilter/iptables。如果有基于传统防火墙的网络流量审计软件，它们会对此短路通信完全失效。必须强制迁移到基于 eBPF 身份标签的安全策略上。

M4.1: Maglev 一致性哈希算法

Theory: 谷歌提出的一致性哈希算法，在后端实例抖动时能够最大化保持已有连接 (会话保持) 的负载均衡算法。

Details: Maglev 预先构建一个大小为质数 (如 65537) 的“查找表 (Lookup Table)”。每个后端服务实例根据其标识符生成特定的伪随机置换序列，交替尝试抢占表中的空槽位。最终生成一个均匀分布的物理映射表，查找表存入 BPF Map。

Trade-off: 构建查找表是一次性 CPU 密集计算。每次 K8s Endpoints 变动时，Cilium Agent 会在用户态增量重新计算并将新查找表下发到 Map 中，确保 $O(1)$ 的内核查找。

M4.4: DSR (Direct Server Return) 路由

Theory: 让后端的容器直接将响应包发送给客户端，绕过负载均衡网关，解决下行流量瓶颈的非对称路由。

Details: 在 DSR 模式下，Cilium 负载均衡器在转发包时，不修改目的 IP，而是将 Real Pod IP 编码进数据包的特定字段 (如 IPv4 Option 或 VxLAN 头)。后端 Pod 收到包后读取此字段，在处理完毕发回响应时，直接把源地址写成 VIP 并发回客户端，无需再流经负载均衡节点。

Trade-off: 必须要保证网络链路支持这种非对称路由，且外层不能有强校验源 MAC 匹配的路由器拦截，配置相对复杂。

M5.1: 身份标识安全标签模型

Theory: 云原生高动态环境下，用静态安全身份标签代替动态多变 IP 地址以保障网络隔离安全的解耦模型。

Details: 传统防火墙为每个 Pod 配置 IP 段规则，但在 K8s 中 IP 频繁变动会导致规则链暴增失效。Cilium 为相同 Label 集合的 Pod 组分配全局一致的 32 位整数 ‘Identity’。在物理发包时，安全标签被编码进 VxLAN 外层头中；目的节点读取标签，与本地策略表比对。

Trade-off: 标签分配由控制面全局协调。如果控制面网络断开，新建容器可能无法获取标签，Cilium 预设了临时应急的安全标签以防止阻断正常通信。

M5.2: IP-to-Identity 地址哈希表映射

Theory: 快速、高效检索目的 IP 所拥有安全标签的高能数据库。

Details: 对于非 Overlay 的直接路由模式，IP 头部无法携带标签。Cilium 在每个节点节点维护一个 ‘BPF_MAP_TYPE_LPM_TRIE’ 类型的 IP-to-Identity 映射表，支持最长前缀匹配。内核包解析阶段，只需一次哈希查找即可判定对方 IP 的身份标识。

Trade-off: 需要全局节点同步该表。当集群规模达到数万 Pod 时，Map 同步开销和内存占用成为挑战。Cilium 仅在本地同步涉及当前通信路径的活跃端点。

M5.3: L3/L4/L7 混合安全策略执行

Theory: 兼顾网卡层极速粗粒度网络过滤与代理层细粒度应用级网关保护的级联架构。

Details: 三层（Identity 隔离）和四层（Port 过滤）安全策略直接在 tc-bpf 内核态执行，开销极低。如果策略包含七层检查（如只允许 GET /api/v1/user 路径），bpf 程序会将该 TCP 链接重定向给运行在宿主机的 Envoy 代理实例，由 Envoy 解析七层协议后再决定是否放行。

Trade-off: 七层策略需要将数据包折回用户态 Envoy，产生上下文切换和内存拷贝损耗。应尽量在 L4 策略拦截大部分流量，仅对核心敏感 API 开启 L7 审计。

M5.4: IPsec 与 WireGuard 加密

Theory: 零侵入、全自动的容器间安全加密数据通道。

Details: Cilium 并不在 BPF 代码内手写加密算法，而是用 eBPF 决定路由和包标记后，触发 Linux 内核原生的 XFRM (IPsec) 框架或 ‘WireGuard’ 虚拟网卡。数据包发出物理网卡前被自动执行硬件加速加密。

Trade-off: 加密算法（如 AES-GCM）会吃掉显著的 CPU 算力，且会导致吞吐下降约 50%。在支持 AES-NI 指令集的物理服务器上开启，能大幅抹平加解密损耗。

Zone T1: eBPF Helper APIs

```
bpf_map_lookup_elem(),      bpf_map_update_elem(),  
  
bpf_redirect(),              bpf_msg_redirect_hash(),  
  
bpf_perf_event_output(),    XDP_DROP,  XDP_REDIRECT,  
XDP_PASS
```

Zone T2: System Network Faults

```
ebpf_verifier_rejection,    conntrack_map_overflow,  
  
socket_redirect_dangling_pointer,  
  
MTU_exceeded_fragmentation,  maglev_hash_collision,  
  
identity_label_sync_split
```

M6.1: Envoy Sidecarless 网格架构

Theory: 消除传统的 Kubernetes Sidecar 注入模式，每个宿主主机只运行一个共享的 Envoy 容器的高效 Mesh。

Details: 相比 Istio 在每个 Pod 强行注入一个 Envoy，Cilium Service Mesh 让宿主主机上的所有 Pod 共享同一个 Envoy。数据包通过 ‘tc-bpf’ 自动捕获并在内核中进行重定向，将其悄无声息地导向该主机的共享 Envoy 代理进行服务治理。

Trade-off: 简化了容器运维管理，省去了数十万容器的内存浪费；但主机 Envoy 一旦崩溃会危及该主机上的所有 Pod 网络，需要高可用的守护服务。

M6.2: Hubble 流量观测与 BPF 元数据追踪

Theory: 云原生网络可观测性显微镜。通过 eBPF 从内核网络路径收集高密度的实时流式分析指标。

Details: Cilium 在网络包经过的关键 tc/XDP 程序中配置了 BPF Ring Buffer。当检测到丢包、策略拦截或大延迟包时，程序将数据包的部分头信息和内部元数据推入 Ring Buffer。用户态的 Hubble 守护进程监听并将其汇总成可视化时延拓扑。

Trade-off: Ring Buffer 写入速率受限。在网络突发泛洪时，为防止内核态被观测逻辑饿死，Hubble 会采取随机抽样（Sampling）和局部丢弃日志策略。